

1. Embedding Model

Trong kiến trúc RAG, chất lượng của bước truy hồi phụ thuộc rất lớn vào embedding model được sử dụng để chuyển đổi văn bản thành vector. Vì hệ thống tư vấn tuyển sinh của đề tài xử lý dữ liệu hoàn toàn bằng tiếng Việt (mô tả ngành học, điều kiện xét tuyển, thông tin tuyển sinh của Trường Đại học Công Thương TP.HCM), việc lựa chọn một embedding model có khả năng biểu diễn ngữ nghĩa tiếng Việt tốt là yêu cầu bắt buộc, vì nếu embedding không phản ánh đúng ngữ nghĩa tiếng Việt, bước truy hồi phía sau dù có thiết kế tốt đến đâu cũng sẽ tìm sai đoạn văn bản liên quan.

Nhóm đã nghiên cứu và cân nhắc hai nhóm giải pháp. Thứ nhất là các mô hình embedding đa ngôn ngữ (multilingual) cung cấp sẵn qua API, tiêu biểu như text-embedding-3 của OpenAI hoặc model / embedding-004 của Google. Ưu điểm của nhóm này là dễ tích hợp, không đòi hỏi hạ tầng máy chủ mạnh để tự vận hành, chỉ cần gọi API là nhận được vector, phù hợp với các dự án có nguồn lực hạ tầng hạn chế. Thứ hai là các mô hình embedding chuyên biệt huấn luyện riêng cho tiếng Việt, tiêu biểu như Vietnamese-sbert hoặc các biến thể fine-tune từ PhoBERT của VinAI Research. Ưu điểm của nhóm này là độ chính xác ngữ nghĩa tiếng Việt thường cao hơn do được huấn luyện chuyên biệt, nhưng đòi hỏi nhóm phải tự triển khai, lưu trữ và duy trì mô hình trên máy chủ riêng, làm tăng độ phức tạp về mặt hạ tầng.

Do điều kiện hạ tầng và tài nguyên phần cứng của đề án có giới hạn, cộng với mong muốn đồng bộ hóa hệ sinh thái công cụ Google Gemini làm mô hình sinh câu trả lời (LLM) ở tầng generation, nhóm quyết định sử dụng mô hình embedding của Google nhằm tận dụng cùng một hệ sinh thái công cụ. Việc này giúp đơn giản hóa quá trình tích hợp (cùng một bộ khóa API, cùng một nhà cung cấp dịch vụ), đồng thời giảm chi phí vận hành hệ thống so với việc phải tự triển khai và duy trì một mô hình embedding riêng.

2. So sánh FAISS và Chroma – lựa chọn Vector DB

Sau khi đã xác định được vai trò của vector DB trong kiến trúc RAG, câu hỏi đặt ra là nên sử dụng công cụ cụ thể nào để triển khai việc lưu trữ và tìm kiếm vector. Trong phạm vi đề tài nhóm tiến hành so sánh hai công cụ phổ biến nhất hiện nay là FAISS và Chroma, dựa trên ba tiêu chí: tốc độ xử lý, mức độ dễ triển khai, và sự phù hợp với quy mô hệ thống.

FAISS là một thư viện mã nguồn mở do Meta phát triển, chuyên về các thuật toán tìm kiếm tương đồng trên vector với hiệu năng cao. Điểm mạnh lớn nhất của FAISS nằm ở tốc độ xử lý, nhờ được tối ưu sâu ở tầng thuật toán, hỗ trợ nhiều cấu trúc chỉ mục (index) khác nhau như IVF (Inverted File Index) hay HNSW (Hierarchical Navigable Small World), FAISS có khả năng tìm kiếm trên hàng triệu vector trong thời gian ngắn, phù hợp với các hệ thống có quy mô dữ liệu lớn trong môi trường sản xuất (production) thực tế. Tuy nhiên, FAISS về bản chất chỉ là một thư viện tính toán thuần túy, không đi kèm cơ chế lưu trữ hay quản lý dữ liệu đầy đủ. Điều này đồng nghĩa với việc để sử dụng được một hệ thống hoàn chỉnh, lập trình viên cần tự xây dựng thêm các thành phần bổ trợ như lưu trữ vector lâu dài (persistence), quản lý danh sách định danh (ID) để ánh xạ vector với văn bản gốc, hay đồng bộ dữ liệu khi có cập nhật, khiến chi phí triển khai ban đầu cao hơn đáng kể.

Ngược lại, Chroma là một hệ cơ sở dữ liệu vector hoàn chỉnh (vector DB), được thiết kế với mục tiêu để sử dụng cho nhà phát triển. Chroma cung cấp sẵn các hàm thao tác ở mức cao như thêm dữ liệu (add) hay truy vấn tìm kiếm (query), đồng thời hỗ trợ tích hợp trực tiếp với các embedding model, cho phép truyền vào văn bản thô và Chroma sẽ tự động gọi mô hình để chuyển đổi thành vector, thay vì lập trình viên phải tự thực hiện từng bước này thủ công như với FAISS. Ngoài ra Chroma cũng hỗ trợ lưu trữ kèm theo metadata. Ví dụ trên ngành học, loại tài liệu, nguồn trích dẫn song song với vector, giúp việc quản lý và truy xuất dữ liệu trở nên trực quan hơn nhiều so với FAISS. Về mặt tốc độ, Chroma không được tối ưu bằng FAISS khi quy mô dữ liệu tăng đến hàng triệu bản ghi, nhưng với quy mô dữ liệu vừa và nhỏ - như kho tri thức của hệ thống tư vấn tuyển sinh

trong đề tài dự kiến chỉ gồm vài trăm đến vài nghìn đoạn văn bản mô tả ngành học và thông tin tuyển sinh của một trường đại học – sự chênh lệch tốc độ giữa hai công cụ gần như không đáng kể trong thực tế sử dụng.

Xét trên cả ba tiêu chí đã nêu, nhóm nhận thấy Chroma là lựa chọn tối ưu và phù hợp hơn cho phạm vi với đề tài. Do quy mô dữ liệu vừa phải, ưu thế về tốc độ của FAISS không tạo ra sự khác biệt thực sự có ý nghĩa trong trải nghiệm sử dụng, trong khi đó Chroma lại giúp giảm thiểu đáng kể khối lượng công việc phải tự xây dựng thêm, cho phép nhóm tập trung nguồn lực và thời gian vào việc phát triển các thành phần cốt lõi khác của hệ thống như xử lý logic truy hồi, thiết kế prompt, và tích hợp mô hình sinh câu trả lời. Vì vậy, nhóm quyết định lựa chọn Chroma làm công cụ vector DB chính thức được triển khai trong đề tài.

Cần lưu ý rằng khi triển khai, do nhóm đã lựa chọn sử dụng mô hình embedding của Google. Chroma cần được cấu hình để sử dụng đúng embedding function trở về mô hình này thay vì sử dụng hàm embedding mặc định đi kèm – nhằm đảm bảo tính nhất quán giữa lựa chọn embedding model đã trình bày và công cụ lưu trữ vector được sử dụng trong toàn bộ hệ thống.

Bảng so sánh:

Tiêu chí	FAISS	Chroma
Nhà phát triển	Meta (Facebook AI)	Chroma (mã nguồn mở độc lập)
Bản chất	Thư viện thuật toán tìm kiếm tương đồng thuần túy	Cơ sở dữ liệu vector đầy đủ
Tốc độ xử lý	Rất nhanh, tối ưu cao cho quy mô lớn (hàng triệu vector)	Đủ nhanh cho quy mô vừa phải

Mức độ dễ triển khai	Cần tự xây dựng thêm lớp lưu trữ, quản lí ID	Có sẵn API thao tác mức cao (add, query)
Tích hợp embedding	Phải tự gọi embedding model, tự truyền vào vector	Tích hợp trực tiếp, tự động embedding văn bản thô
Hỗ trợ metadata	Không hỗ trợ sẵn, phải tự làm	Hỗ trợ sẵn, lưu kèm vector
Phù hợp quy mô	Lớn (production, hàng triệu bản ghi)	Nhỏ - vừa (phù hợp với đề tài)

3. Chiến lược Chunking văn bản

Trước khi có thể đưa vào embedding và lưu trữ trong vector DB, các tài liệu gốc của hệ thống vốn có độ dài lớn, chẳng hạn như đề án tuyển sinh hay tài liệu mô tả chi tiết chương trình đào tạo của từng ngành thì phải cần được chia nhỏ thành các đoạn văn bản có kích thước phù hợp, gọi là quá trình chunking (phân đoạn văn bản). Việc chunking hợp lý có ảnh hưởng trực tiếp đến độ chính xác của bước truy hồi phía sau: nếu chunking quá lớn, vector đại diện cho đoạn đó sẽ mang nhiều thông tin lẫn lộn từ nhiều chủ đề khác nhau, làm giảm độ chính xác khi so khớp với câu hỏi cụ thể của người dùng; ngược lại nếu chunking quá nhỏ, một thông tin có thể bị cắt rời khỏi phần ngữ cảnh cần thiết để hiểu trọn vẹn ý nghĩa của câu đó.

Có nhiều chiến lược chunking được sử dụng phổ biến trong thực tế:

- Chunking theo độ dài cố định (fixed-size chunking) chia văn bản thành các đoạn có số lượng kí tự hoặc token bằng nhau, đây là cách đơn giản nhất để triển khai, nhưng có nhược điểm lớn là dễ cắt ngang giữa các câu hoặc ý đang trình bày dở dang làm mất đi tính trọn vẹn của thông tin.
- Chunking theo câu (sentence-based chunking) khắc phục một phần nhược điểm này bằng cách đảm bảo mỗi chunk luôn kết thúc trọn vẹn ở ranh

giới một câu hoàn chỉnh, không bị cắt giữa chừng.

Tuy nhiên, chiến lược được nhóm đánh giá phù hợp với đặc điểm dữ liệu của đề tài là chunking theo ngữ nghĩa (semantic chunking) là chia văn bản theo các đơn vị nội dung có ý nghĩa hoàn chỉnh, chẳng hạn như mỗi ngành học hoặc mỗi mục thông tin riêng biệt (điều kiện xét tuyển, tổ hợp môn thi, chỉ tiêu tuyển sinh, học phí,..) được tách thành một chunk riêng. Do dữ liệu tuyển sinh của trường vốn đã có cấu trúc rõ ràng, được tổ chức theo từng mục trong các tài liệu gốc (đề án tuyển sinh, website trường), việc chunking theo đúng ranh giới ngữ nghĩa tự nhiên này giúp mỗi đoạn văn bản giữ được sự trọn vẹn về mặt nội dung, đồng thời phản ánh đúng cách con người tổ chức và tra cứu thông tin.

Bên cạnh việc lựa chọn chiến lược chia đoạn, nhóm áp dụng thêm kỹ thuật chồng lấn (overlap) giữa các chunk liên kế nhau, tức là một phần nội dung nhỏ ở cuối chunk trước được lặp lại ở đầu chunk sau. Kỹ thuật này nhằm hạn chế trường hợp một thông tin quan trọng nằm đúng vào vị trí ranh giới giữa hai chunk, dẫn đến việc bị tách rời hoàn toàn khi chia đoạn, khiến cả hai chunk khi được truy hồi riêng lẻ đều không thể hiện đầy đủ ý trọn vẹn của thông tin đó.

VÍ DỤ MINH HỌA CHUNKING THEO NGŨ NGHĨA (SEMANTIC CHUNKING) CÓ OVERLAP

1. VĂN BẢN GỐC (Đoạn mô tả ngành CNTT)

Ngành Công nghệ thông tin (CNTT) đào tạo sinh viên có kiến thức và kỹ năng về lập trình, hệ thống thông tin, cơ sở dữ liệu, mạng máy tính và trí tuệ nhân tạo. Sinh viên được trang bị nền tảng vững chắc về toán học, thuật toán và khoa học máy tính, đồng thời được thực hành trên các công nghệ hiện đại.

Chương trình học được thiết kế theo hướng ứng dụng, cập nhật xu hướng công nghệ mới như điện toán đám mây, dữ liệu lớn và an toàn thông tin. Sinh viên có cơ hội tham gia các dự án thực tế, thực tập tại doanh nghiệp và nghiên cứu khoa học.

Cơ hội việc làm rộng mở với các vị trí như lập trình viên, kỹ sư phần mềm, chuyên viên phân tích dữ liệu, quản trị hệ thống, chuyên gia an ninh mạng tại các công ty công nghệ, tổ chức và cơ quan nhà nước.

CHÚ THÍCH

-  Nội dung chính của chunk
-  Phần overlap (chồng lặp)

THÔNG SỐ MINH HỌA

- Chiến lược: Semantic Chunking theo các mục nội dung tự nhiên
- Kích thước chunk: ~100–120 từ
- Overlap: ~20–30 từ (phần nội dung cuối của chunk trước lặp lại ở đầu chunk sau)

2. KẾT QUẢ CHIA CHUNK CÓ OVERLAP

CHUNK 1 – Giới thiệu chung

Ngành Công nghệ thông tin (CNTT) đào tạo sinh viên có kiến thức và kỹ năng về lập trình, hệ thống thông tin, cơ sở dữ liệu, mạng máy tính và trí tuệ nhân tạo. Sinh viên được trang bị nền tảng vững chắc về toán học, thuật toán và khoa học máy tính, đồng thời được thực hành trên các công nghệ hiện đại.

Chương trình học được thiết kế theo hướng ứng dụng, cập nhật xu hướng công nghệ mới như điện toán đám mây, dữ liệu lớn và an toàn thông tin.

~115 từ

CHUNK 2 – Chương trình đào tạo

Chương trình học được thiết kế theo hướng ứng dụng, cập nhật xu hướng công nghệ mới như điện toán đám mây, dữ liệu lớn và an toàn thông tin.

Sinh viên được trang bị kiến thức chuyên sâu và kỹ năng thực hành thông qua các môn học từ cơ bản đến nâng cao. Sinh viên có cơ hội tham gia các dự án thực tế, thực tập tại doanh nghiệp và nghiên cứu khoa học.

~110 từ

CHUNK 3 – Cơ hội việc làm

Sinh viên có cơ hội tham gia các dự án thực tế, thực tập tại doanh nghiệp và nghiên cứu khoa học.

Cơ hội việc làm rộng mở với các vị trí như lập trình viên, kỹ sư phần mềm, chuyên viên phân tích dữ liệu, quản trị hệ thống, chuyên gia an ninh mạng tại các công ty công nghệ, tổ chức và cơ quan nhà nước.

~100 từ

OVERLAP
(~25 từ)

OVERLAP
(~25 từ)



Nhờ phần overlap (~20–30 từ), các thông tin nằm ở ranh giới giữa các chunk không bị tách rời hoàn toàn. Khi truy hồi, mỗi chunk đều giữ được ngữ cảnh cần thiết, giúp mô hình hiểu đầy đủ và trả lời chính xác hơn.

4. Prompt Engineering cho trích dẫn nguồn

Một trong những yêu cầu quan trọng của hệ thống là câu trả lời do chatbot sinh ra không chỉ cần chính xác về mặt nội dung mà còn phải có khả năng trích dẫn rõ thông tin được sử dụng, nhằm tăng độ tin cậy của hệ thống và cho phép người dùng tự kiểm chứng lại thông tin nếu cần thiết. Để đạt được điều này, nhóm thiết kế prompt gửi đến mô hình sinh câu trả lời theo ba nguyên tắc chính:

Thứ nhất, mỗi đoạn văn bản (chunk) khi được lưu trữ trong vector DB đều được gắn kèm thông tin về nguồn gốc cụ thể, chẳng hạn tên tài liệu gốc và mục cụ thể mà đoạn văn bản đó được trích ra. Ví dụ như: “Nguồn: Đề án tuyển sinh năm 2026, mục 3.2 Điều kiện xét tuyển”. Khi một chunk được truy hồi để đưa vào ngữ cảnh trả lời cho một câu hỏi cụ thể, nguồn thông tin này được giữ nguyên và kèm theo vào prompt gửi cho mô hình sinh câu trả lời, không bị tách rời khỏi

nội dung.

Thứ hai, phần chi tiết dẫn hệ thống (system prompt) yêu cầu rõ ràng mô hình phải trích dẫn nguồn ngay sau mỗi thông tin được sử dụng trong câu trả lời, theo một định dạng thống nhất đã quy định trước. Đồng thời, chỉ dẫn này yêu cầu mô hình chỉ được trả lời dựa trên nội dung ngữ cảnh đã được cung cấp, tuyệt đối không được tự suy diễn hay bổ sung thêm thông tin nằm ngoài phạm vi ngữ cảnh đó. Trong trường hợp ngữ cảnh được truy hồi không chứa đủ thông tin để trả lời câu hỏi của người dùng, mô hình được yêu cầu phản hồi rằng hệ thống chưa có đủ thông tin về vấn đề đó, thay vì tự tạo ra một câu trả lời không có căn cứ thực tế, đây chính là cơ chế trực tiếp giúp giảm hiện tượng ảo giác.

Thứ ba, nhóm bổ sung thêm một số ví dụ mẫu (few-shot examples) ngay trong nội dung prompt, minh họa cụ thể cách một câu hỏi nên được trả lời kèm theo trích dẫn đúng định dạng mong muốn. Việc đưa ví dụ mẫu trực tiếp vào prompt giúp mô hình sinh ra kết quả nhất quán hơn qua nhiều lượt hỏi đáp khác nhau, giảm thiểu tình trạng định dạng trích dẫn bị thay đổi tùy tiện giữa các câu trả lời.



THIẾT KẾ PROMPT MẪU (PROMPT TEMPLATE)

1. SYSTEM PROMPT

Bạn là trợ lý ảo thông minh của HUIT (Trường Đại học Công nghiệp TP.HCM). Nhiệm vụ của bạn là trả lời câu hỏi của sinh viên **DỰA TRÊN DUY NHẤT** ngữ cảnh được cung cấp.

QUY TẮC TRÍCH DẪN & AN TOÀN:

- Trích dẫn nguồn ngay sau mỗi thông tin được sử dụng trong câu trả lời.
- Định dạng: [Nguồn: Tên tài liệu/Mục].
- Tuyệt đối không tự suy diễn, bịa đặt ngoài ngữ cảnh.
- Nếu không đủ thông tin, phản hồi: "Hiện tại hệ thống chưa có đủ thông tin về vấn đề này."

2. NGỮ CẢNH TRUY HỒI (RETRIEVED CONTEXT)

[Chunk 1] Nguồn: Sổ tay Sinh viên HUIT - Mục 2.1: Chương trình đào tạo
Chương trình đào tạo đại học hệ chính quy áp dụng cho khóa tuyển sinh từ năm 2024 bao gồm 150 tín chỉ tích lũy, được chia thành các khối kiến thức: Đại cương (45 tín chỉ), Cơ sở ngành (60 tín chỉ) và Chuyên ngành (45 tín chỉ). Sinh viên phải hoàn thành tất cả các học phần bắt buộc và tự chọn.

[Chunk 2] Nguồn: Quy chế Đào tạo HUIT - Chương IV: Điều kiện tốt nghiệp
Điều kiện để sinh viên được công nhận tốt nghiệp bao gồm:

- (a) Tích lũy đủ số tín chỉ theo quy định của chương trình đào tạo,
- (b) Điểm trung bình chung tích lũy toàn khóa đạt từ 2.0 trở lên (trên thang điểm 4.0),
- (c) Hoàn thành các học phần Giáo dục Quốc phòng - An ninh và Giáo dục Thể chất,
- (d) Không bị truy cứu trách nhiệm hình sự hoặc đang trong thời gian kỷ luật ở mức đình chỉ học tập.

3. CÂU HỎI NGƯỜI DÙNG (USER QUERY)

Sinh viên khóa 2024 cần tích lũy bao nhiêu tín chỉ để tốt nghiệp HUIT?

KẾT QUẢ ĐẦU RA (MODEL RESPONSE)

Dựa trên Sổ tay Sinh viên HUIT (Mục 2.1), tổng số tín chỉ tích lũy cần thiết để sinh viên khóa 2024 tốt nghiệp là 150 tín chỉ.

[Nguồn: Sổ tay Sinh viên HUIT - Mục 2.1].

Ngoài ra, theo Quy chế Đào tạo HUIT (Chương IV), sinh viên cũng cần đạt điểm trung bình chung tích lũy toàn khóa từ 2.0 trở lên, hoàn thành các học phần Giáo dục Quốc phòng - An ninh và Giáo dục Thể chất.

[Nguồn: Quy chế Đào tạo HUIT - Chương IV].

5. Kỹ thuật đánh giá Hallucination

Để đảm bảo chất lượng của hệ thống chatbot RAG trước khi đưa vào sử dụng

thực tế, nhóm cần có phương pháp đánh giá định lượng mức độ trung thực của câu trả lời được sinh ra, thay vì chỉ dựa trên đánh giá cảm tính, chủ quan. Một số chỉ số được nhóm nghiên cứu và áp dụng cho mục đích này.

Chỉ số độ trung thực (faithfulness) đo lường mức độ câu trả lời được sinh ra có thực sự dựa trên ngữ cảnh đã truy hồi được hay không. Chỉ số này thường được tính bằng cách tách câu trả lời của mô hình thành các luận điểm nhỏ hơn (claims), sau đó kiểm tra từng luận điểm một có được nội dung trong ngữ cảnh cung cấp hỗ trợ hay không, tỷ lệ số luận điểm được hỗ trợ trên tổng số luận điểm chính là điểm faithfulness.

Chỉ số độ liên quan của câu trả lời (answer relevancy) đo lường mức độ câu trả lời có thực sự giải đáp đúng trọng tâm câu hỏi được người dùng đặt ra hay không, tránh tình trạng câu trả lời tuy đúng về mặt sự kiện nhưng lại lạc đề, không giải quyết đúng vấn đề người dùng quan tâm. Chỉ số này thường được tính thông qua độ tương đồng cosine giữa vector embedding của câu hỏi gốc và vector embedding của câu trả lời được sinh ra.

Ngoài hai chỉ số đánh giá tăng sinh câu trả lời (generation) nêu trên, nhóm cũng đánh giá riêng chất lượng của tầng truy hồi (retrieval) thông qua hai chỉ số là độ chính xác truy hồi (context precision trong - số các đoạn văn bản được truy hồi, bao nhiêu phần trăm thực sự liên quan đến câu hỏi) và độ bao phủ truy hồi (context recall - trong số các đoạn văn bản thực sự cần thiết để trả lời đúng câu hỏi, hệ thống truy hồi được bao nhiêu phần trăm). Việc tách riêng đánh giá cho hai tầng retrieval và generation giúp nhóm xác định chính xác nguồn gốc của lỗi khi hệ thống trả lời sai lỗi đến từ việc tìm sai tài liệu (retrieval) hay từ việc mô hình sinh sai dù đã có tài liệu đúng (generation), từ đó có hướng khắc phục phù hợp cho từng trường hợp.

Để hỗ trợ việc tính toán các chỉ số nêu trên một cách có hệ thống và nhất quán, nhóm dự kiến sử dụng thư viện mã nguồn mở RAGAS (Retrieval Augmented Generation Assessment), vốn được thiết kế chuyên biệt cho mục đích đánh giá

các hệ thống RAG, đã tích hợp sẵn công thức tính cho cả bốn chỉ số đã nêu, giúp nhóm không cần tự cài đặt lại từ đầu. Quy trình đánh giá thực tế được nhóm thực hiện bằng cách chuẩn bị trước một bộ câu hỏi kiểm thử kèm theo câu trả lời chuẩn đã biết trước (ground truth), sau đó đưa lần lượt các câu hỏi này qua hệ thống RAG đã xây dựng và so sánh kết quả câu trả lời thực tế sinh ra với câu trả lời chuẩn theo các chỉ số đã trình bày. Kết quả đánh giá định lượng này chính là cơ sở khách quan để nhóm nhận diện điểm yếu và cải thiện chất lượng hệ thống trước khi tiến hành kiểm thử với người dùng thực tế ở giai đoạn sau.

[Hình ảnh cần bổ sung: Bảng kết quả mẫu (có thể dùng số liệu giả định minh họa cách trình bày, sẽ thay bằng số liệu thật khi có kết quả thực nghiệm) gồm các cột: Câu hỏi test, Faithfulness score, Answer Relevancy score, Context Precision, Context Recall — giúp hình dung cách RAGAS xuất kết quả]